Document number: P0240R0
Audience: WG21

Matt Calabrese
2016-02-12

Why I want Concepts, but why they should come later rather than sooner

Abstract

This paper is a response to P0225R0, "Why I want Concepts, and why I want them sooner rather than later."[1] The author presents an opinionated view, though with the opposing perspective -- that Concepts is not yet ready for C++ and that it would be a mistake to prematurely include it as a part of C++17.

Contents

In brief

For over a decade, people have investigated bringing direct, language-level support for generic programming to C++. We've seen multiple approaches that have been able to directly represent differing subsets of the vocabulary of the paradigm. Most recently, with the Concepts TS, we have an approach that represents constraints via usage patterns, and provides concept-based overloading.

However, what is currently provided by the Concepts TS is not complete. First, as has been pointed out by P0225R0, we do not yet have a conceptified version of the standard library. Unlike P0225R0, however, I do *not* consider this something that can be written off. Having a conceptified standard library is important not just to prove out the feature, but also to serve as an example to C++ developers as to how Concepts should be used, down to the subtleties that are undoubtedly required by the standard library. As well, if we fail to ship a conceptified standard library with the initial integration of Concepts into C++, we will likely see common concepts that the standard library should provide being invented and reinvented with varying levels of quality by various development teams. This is not something that we should be inviting.

In addition to this, there is one feature that many (though admittedly not all) generic programmers, feel is extremely important to any Concept implementation, and yet is currently left out. That feature is the ability for a constrained template to automatically and immediately validate its definition at the time of writing, prior to instantiation, by verifying that the body only uses functionality that is allowed by the constraints, thereby making sure that the writer of the template neither accidentally uses functionality outside of its specified constraints, nor accidentally under-constrains their template. Without this feature, constraints provide a mostly superficial advantage over existing techniques, even though those superficial changes are, as Ville has put it, "alluring."

A lack of a feature in other contexts is not normally a problem -- we can usually just add a missing feature in a future standard with little or no harm done. However, with respect to the checking of constrained template definitions, there is strong reason to believe that it would not be possible to add definition checking to Concepts in a way that is meaningful without serious breaking changes. The reliability of the checking of definitions in C++0x concepts, which notably had a working implementation that included definition checking by way of ConceptGCC, stemmed from the fine-grained and restricted way of specifying requirements through pseudo-signatures, the restriction of definitions to only directly viewing the result and parameter types of those pseudo-signatures, and the fact that name lookup within a constrained template was modified to match the constraints such that the associated functions of the concepts involved were discovered and preferred.[2] These latter points are essential in preventing errors during instantiation and as a means to provide consistency of lookup of associated functions. Further, during

the implementation of a constrained standard library for C++0x, it was discovered that most constraints that were declared were actually incorrect in very subtle ways, despite the fact that those constraints were written by experts.

For the above reasons, it is very difficult to imagine that the average programmer will be able to correctly write constraints without aid. In addition, it is likely that if we eventually do get definition checking, name lookup within constrained templates would have to change, breaking existing code considerably even if the developer managed to write a constraint that was truly correct. This is the unfortunate reality.

In a very recent correspondence with Doug Gregor, one of the principle people behind C++0x concepts and the primary developer of ConceptGCC, I asked him for his own perspective on this issue. The following was his response, quoted with permission:

> "I am completely convinced that putting concepts into the C++ standard without separate type checking of template definitions means that we will never get separate type checking of template definitions in standard C++. Within a close approximation, all of the constrained templates written before that point will have incorrect or incomplete requirements, leaving no path forward except to having two constraint systems. Moreover, it's extremely likely that the design of concepts does not admit separate type checking of template definitions in any real way (despite inevitable protests to the contrary)."
>
> -- *Doug Gregor*

Who I am

For some context as to why I consider myself qualified to make the assessments in this paper -- I have been programming in C++ for about 15 years and have been an active member of the Boost community since around 2004. Specifically, I write a lot of code using the generic programming paradigm. My most relevant experience to the topic at hand is that I have worked with C++0x concepts prior to them being pulled from C++11 and implemented an EDSL[3] in standard C++ for directly representing and checking the concepts that were presented in the last working paper that contained C++0x concepts, N2914[4]. I am knowledgeable of many of the subtleties of concepts as a language feature, including what is necessary for meaningful checking of constrained template definitions.

Though I have a solid understanding of previous efforts, I was never, myself, involved with any of its associated proposals. While I am skeptical regarding certain aspects of the Concepts TS, I am fairly removed from the controversy surrounding the pulling of concepts from C++11 and the move towards the current effort. My views are my own conclusions that I've come to as an observer over the course of many years. I have profound respect for all people involved with the various attempts to bring more direct support for the generic programming paradigm to C++, though I do have serious concerns about the current state of the Concepts TS that make me worried about its potential inclusion in C++17.

Do we care about checking constrained-template definitions?

Though P0225R0 defends the inclusion of Concepts in C++ even without standard library support, which is something I strongly disagree with, my main concern is that the current specification likely rules out us ever getting the checking of constrained template definitions in the language, short of deprecating Concepts and reintroducing a new facility that is properly suited for the task at some point considerably further in the future than the standard after C++17. Because of this, before considering Concepts for inclusion in C++17, we should first be sure that everyone is either aware of the very real likelihood of the Concepts TS being unsatisfactory for definition checking, or the community needs to decide once and for all that definition checking isn't important and that we are willing to sacrifice it in order to get concepts into the language a couple of years early.

Upon speaking with various C++ developers who are interested in generic programming informally, both inside and outside of the committee, I've discovered that people fall into one of a few categories:

- Those who want concepts as a language feature and consider definition checking to be important, with the impression that definition checking will be able to come without significantly modifying or redesigning Concepts
- Those who want concepts as a language feature and consider definition checking to be important, with the belief that Concepts as-is is not suitable for definition checking
- Those who want concepts as a language feature and don't consider definition checking to be important
- Those who do not care about concepts as a language feature

Of the above list, the most troubling thing to me is (anecdotally) how many people fall into the first category, despite the fact that some of those who understand the issues, including a chief driver of C++0x concepts and the implementor of ConceptGCC, have explicitly stated that they do not see the current Concepts TS as being at all capable of providing proper definition checking. At the very least, before considering Concepts for inclusion in C++17, that issue needs to be resolved. People need to be fully aware of what it is they are getting and what it is they are likely ruling out if they are expected to make an informed decision.

While I personally feel that definition checking is extremely important, as do others, there are also notable people in the generic programming community who do not consider it quite so important and are willing to sacrifice the chance of us ever getting it.

Complicated template errors will remain if we do not get definition checking

One of the goals of a concepts language feature is to reduce or eliminate those verbose template instantiation backtraces that we all know and love when we encounter a compile error as a part of a template library. The Concepts TS helps with one side of this problem by making it easier to specify constraints at the interface level, but much of the problem still remains. Keep in mind that for years we've had access to SFINAE exploits that allow such checks, and even when they are used by the "too clever" programmer, as Ville refers to them, crazy template errors still remain when in the body of a constrained template. Whether those constraints are specified with SFINAE explicitly or via concepts without definition checking cannot considerably change that.

This is true for a number of reasons. First, when a programmer developing generic code gets constraints incorrect, it is the user of that template who suffers, as they will once again see a horrible template backtrace at the time they "correctly" use the template (correctly in the sense that they met the specified requirements). Additionally, even if the constraints are specified correctly, actually writing a template in a way that strictly uses the functionality specified in the constraints in a meaningful way is *incredibly* subtle. In practice, this means explicitly casting results of associated functions and also invoking those functions through traits and qualified calls and generally avoiding direct use of ADL.

Writing constrained templates is exceedingly difficult

Writing constraints is difficult -- so difficult that even experts fail to properly specify constraints. Again, quoting a recent conversation with Doug Gregor, regarding the conceptification of the standard library for C++0x:

> “...when we added type checking of template definitions, there was a *huge* reckoning in the implementation of the concepts-based standard library. Only the most trivial constrained function templates had a correct set of requirements. Most had missing requirements of some form or other. We had that reckoning before putting forward serious proposals for a concept-based standard library, so it happened mostly outside of the committee process.”
>
> *-- Doug Gregor*

If we are to be concerned with writing *correct* generic code, and not just code that provides some superficial advantages over SFINAE and tag-dispatch (though that is also important), then we really do need definition checking. Given that even experts have trouble correctly specifying seemingly simple constraints, I do not think it is wise to assume that the average programmer is capable of doing so, regardless of confidence and the illusion of it occasionally being trivial. Definition checking is important to experienced generic programmers and to newcomers alike.

It is unlikely that the Concepts TS as-is will be able to do definition checking

For those who are interested in the gritty details of why definition checking wouldn't really be feasible with the Concepts TS without considerable breaking changes, a couple of very subtle examples are described in a discussion that took place in 2012 on the Boost Developers mailing list comparing pseudo-signatures and usage patterns.[5] I encourage those who are interested in the topic to read it, as the details are out of scope of this paper. The conversation consisted of myself and several key people who were involved with Concepts of the past and of the present, notably including Dave Abrahams, Doug Gregor, Andrew Sutton, Larisse Voufo, and Jeremiah Willcock. Sadly, while the issues were understood and had been handled with C++0x concepts, and the reasons why it would be difficult to bring such checking to what would become the Concepts TS were known, little has changed and we are still at a point where there is no clear path forward toward proper definition checking in the Concepts TS. Definition checking is not simply something that can be added later. It is a feature that needs consideration throughout the design of a concepts language feature as it directly influences how constraints need to be represented.

Should the design of Concepts be validated by applying them to the library?

Yes!

Conclusion

In the very words of P0225R0, bringing Concepts into C++17 requires a "leap of faith". It is suggested that the risk of this leap is worthwhile in order to bring Concepts to programmers who are aching to use it. In actuality, no such leap is necessary. Programmers already can use Concepts in practice if they are willing to deal with a feature that is likely to evolve more quickly and be more willing to make breaking changes than C++ is. Let the Concepts TS continue evolving. This allows it to make large, breaking changes if and when such changes need to be made. For those who believe that the checking of constrained template

definitions is important, the breakages likely would be vast, and we should not have to hesitate to make them. We should be free to do what makes the language the best that it can be with respect to the direct representation of the generic programming paradigm.

Though I am not traditionally one who is conservative in these matters, in order to avoid the very real risk of painting ourselves into a corner regarding definition checking at a point in time where many people feel that it is important, we should hold off on Concepts. The concern is not FUD, but rather it is the reality of the complexities of definition checking in C++. The Concepts TS will get usage experience regardless, and in the upcoming years we should have both a constrained version of the standard library and a better answer as to whether or not this approach really is capable of providing a path towards the checking of constrained template definitions. At that point, we will be in a better position to safely bring Concepts into C++.

References

[1] Ville Voutilainen: "Why I want Concepts, and why I want them sooner rather than later" P0225R0 https://isocpp.org/files/papers/p0225r0.html

[2] Douglas Gregor and Jeremy Siek: "Implementing Concepts" N1484 http://www.crest.iu.edu/publications/prints/2005/gregor05:implementing_concepts.pdf

[3] Matt Calabrese: "Boost.Generic: Concepts without Concepts" https://raw.githubusercontent.com/boostcon/2011_presentations/master/thu/Boost.Generic.pdf

[4] "Working Draft, Standard for Programming Language C++ N2914 http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2009/n2914.pdf

[5] Various Participants "concepts: pseudo-signatures vs. usage patterns" http://thread.gmane.org/gmane.comp.lib.boost.devel/234959